

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
Course Code	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week3 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
AssignmentNumber:6.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures.	Week3 - Wednesday																		

- To evaluate the correctness, readability, and completeness of AI-generated Python code.

Lab Outcomes (LOs)

After completing this lab, students will be able to:

- Use AI tools to generate and complete Python class definitions and methods.
- Understand and assess AI-suggested loop constructs for iterative tasks.
- Generate and evaluate conditional statements using AI-driven prompts.
- Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method display_details() to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

- A Python class with a constructor (__init__) and a display_details() method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code.

PROMPT: Write a Python Student class with name, roll number, and branch.

Add a constructor and a display_details() method.

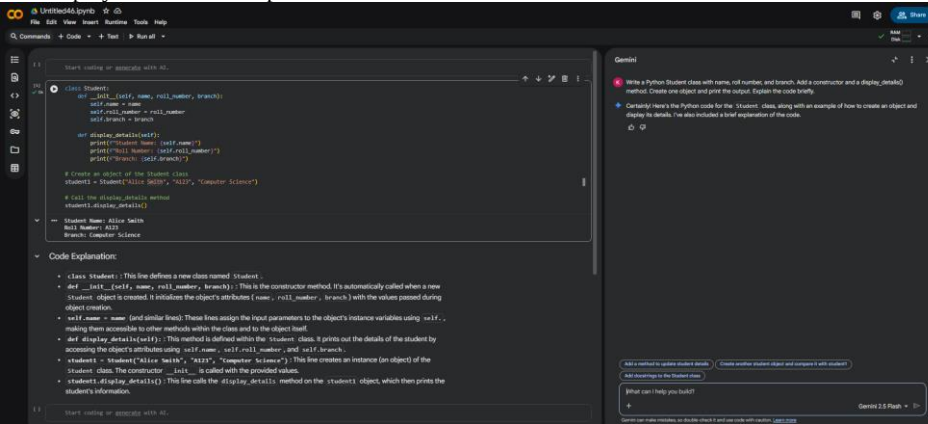
Create one object and print the output.

Explain the code briefly.

EXPLANATION: The Student class is used to store student details like name, roll number, and branch.

The init method initializes these values when an object is created.

The display details method prints the student information on the screen.



Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.

- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

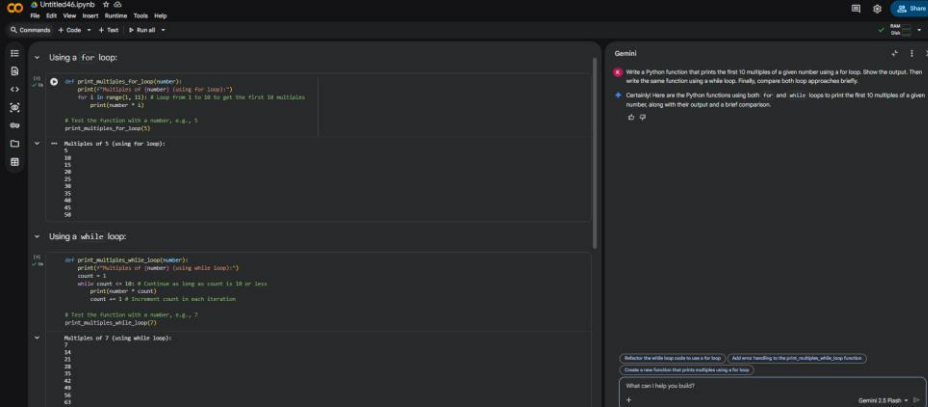
Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

PROMPT: Write a Python function that prints the first 10 multiples of a given number using a for loop.

Show the output.

Then write the same function using a while loop.



EXPLANATION: The `for` loop is short and easy because the number of iterations is fixed. The `while` loop needs a counter and condition, so it is slightly longer. Both loops produce the same output and are correct.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested `if-elif-else` conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

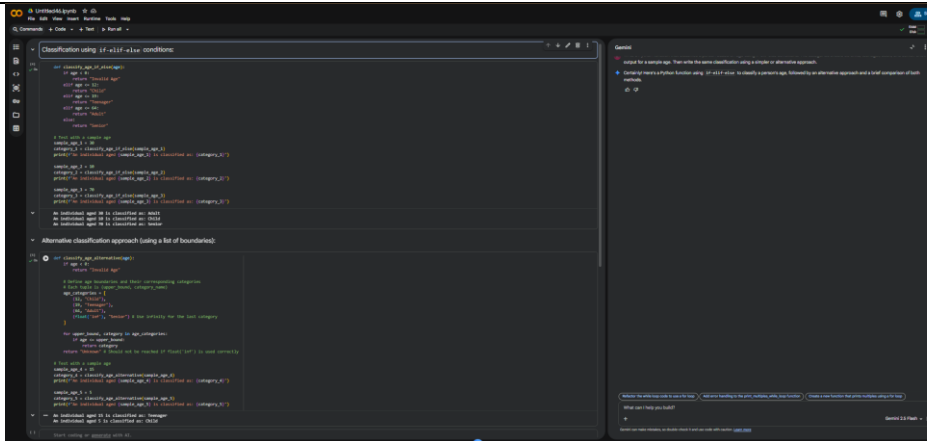
- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

PROMPT: Write a Python function that classifies a person based on age using `if-elif-else` conditions.

The categories should be child, teenager, adult, and senior.

Show the output for a sample age.

Then write the same classification using a simpler or alternative approach.



EXPLANATION: The if-elif-else method checks age step by step and assigns a group. The simplified approach uses a list of limits and loops to find the correct group. Both methods give the same result, but the first is easier for beginners.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

PROMPT: Write a Python function `sum_to_n` that calculates the sum of the first n natural numbers using a for loop.

Show the output for a sample value.

Then write the same function using a while loop or a mathematical formula.

1. Using a for loop:

```
10 def sum_to_n_for_loop(n):
11     if n < 0:
12         return "Input must be a non-negative integer"
13     total = 0
14     for i in range(1, n + 1):
15         total += i
16     return total
17
18 # Sample value
19 sample_n_for = 10
20 result_for = sum_to_n_for_loop(sample_n_for)
21 print(f"The sum of the first {sample_n_for} natural numbers using a for loop is: {result_for}")
22
23 sample_n_for_neg = -5
24 result_for_neg = sum_to_n_for_loop(sample_n_for_neg)
25 print(f"The sum of the first {sample_n_for_neg} natural numbers using a for loop is: {result_for_neg}")
```

✓ The sum of the first 10 natural numbers using a for loop is: 55
The sum of the first -5 natural numbers using a for loop is: Input must be a non-negative integer

2. Alternative approaches:

a) Using a while loop:

```
10 def sum_to_n_while_loop(n):
11     if n < 0:
12         return "Input must be a non-negative integer"
13     total = 0
14     counter = 1
15     while counter <= n:
16         total += counter
17         counter += 1
18     return total
19
20 # Sample value
21 sample_n_while = 7
22 result_while = sum_to_n_while_loop(sample_n_while)
23 print(f"The sum of the first {sample_n_while} natural numbers using a while loop is: {result_while}")
```

✓ The sum of the first 7 natural numbers using a while loop is: 28

b) Using a mathematical formula:

```
10 def sum_to_n_formula(n):
11     if n < 0:
12         return "Input must be a non-negative integer"
13     # The formula for the sum of the first n natural numbers is n * (n + 1) / 2
14     return n * (n + 1) // 2 # Use integer division for natural numbers
15
16 # Sample value
17 sample_n_formula = 100
18 result_formula = sum_to_n_formula(sample_n_formula)
19 print(f"The sum of the first {sample_n_formula} natural numbers using a mathematical formula is: {result_formula}")
```

✓ The sum of the first 100 natural numbers using a mathematical formula is: 5050

EXPLANATION: The **for loop** adds numbers one by one and is easy to understand. The while loop works similarly but needs manual control of the counter. The formula method is fastest and does not use loops.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

PROMPT: Create a simple BankAccount class in Python.

The class should have methods:

- deposit()
- withdraw()
- check_balance()

Show how money is deposited and withdrawn and display the updated balance.

```

class BankAccount:
    def __init__(self, initial_balance=0):
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be negative.")
        self.balance = initial_balance
        print(f"Account created with initial balance: ${self.balance:.2f}")

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be positive.")
            return
        self.balance += amount
        print(f"Deposited: ${amount:.2f}")
        self.check_balance()

    def withdraw(self, amount):
        if amount <= 0:
            print("Withdrawal amount must be positive.")
            return
        if amount > self.balance:
            print("Insufficient funds.")
            self.balance -= amount
            print(f"Withdrew: ${amount:.2f}")
            self.check_balance()
        else:
            self.balance -= amount
            print(f"Withdrew: ${amount:.2f}")
            self.check_balance()

    def check_balance(self):
        print(f"Current balance: ${self.balance:.2f}")

# --- Demonstration ---
print("\n--- Creating a new bank account ---")
my_account = BankAccount(100.00)

print("\n--- Depositing money ---")
my_account.deposit(50.75)
print(f"Deposited: $50.75")
print(f"Current balance: $150.75")

print("\n--- Withdrawing money ---")
my_account.withdraw(20.00)
print(f"Withdrew: $20.00")
print(f"Current balance: $130.75")

print("\n--- Attempting to withdraw more than available ---")
my_account.withdraw(150.00)
print(f"Insufficient funds.")
print(f"Current balance: $130.75")

print("\n--- Depositing a negative amount ---")
my_account.deposit(-10.00)
print(f"Deposit amount must be positive.")

print("\n--- Final balance ---")
my_account.check_balance()

# --- Output ---
--- Creating a new bank account ---
Account created with initial balance: $100.00

--- Depositing money ---
Deposited: $50.75
Current balance: $150.75

--- Withdrawing money ---
Withdrew: $20.00
Current balance: $130.75

--- Attempting to withdraw more than available ---
Insufficient funds.
Current balance: $130.75

--- Depositing a negative amount ---
Deposit amount must be positive.

--- Final balance ---
Current balance: $130.75

```

EXPLANATION:

The Bank Account class stores account details and balance.

The deposit method adds money, and withdraw removes money if balance is sufficient.

The check balance method shows the current balance.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.